

PLAN DE TESTS

Projet : Dequantification par arbre binaire

Module de reconstruction de signal quantifié

Référence du document :	D7
Version du document :	1.0
Date du document :	2026
Auteur(s) :	Équipe projet
Mots clés :	Tests, Recette, Intégration, Unitaire, Validation, Dequantification

1. Introduction

Ce document constitue le plan de tests du module de dequantification par arbre binaire. Il définit l'ensemble des tests à réaliser pour garantir que le logiciel respecte le cahier des charges, couvre toutes les fonctionnalités implémentées et vérifie la robustesse de chaque composant.

Le plan de tests est organisé en trois niveaux complémentaires : les tests fonctionnels (validation du comportement global), les tests d'intégration (validation des interfaces entre modules) et les tests unitaires (validation de chaque fonction individuellement).

1.1 Objectifs et méthodes

Le présent plan de tests vise à :

- Vérifier que la reconstruction du signal à partir d'une série quantifiée produit des résultats corrects et cohérents avec le signal original.
- Valider le bon fonctionnement de la structure arborescente de dequantification, notamment la gestion des noeuds, les transitions d'histogramme et l'élagage.
- Assurer la robustesse du programme face à des données d'entrée variées (petits fichiers, grands fichiers, signaux répétitifs).
- Vérifier la conformité de l'affichage SDL2 avec les états attendus (noeuds vivants, morts, solutions).

1.2 Documents de référence

- Cahier des charges — Dequantification par arbre binaire
- Spécification de l'algorithme construire() — pseudo-code fourni
- Documentation SDL2 — <https://wiki.libsdl.org/>
- Code source : tree.c/.h, histogram.c/.h, display.c/.h, series.c/.h, main.c

2. Guide de lecture

2.1 Maîtrise d'œuvre

L'équipe de développement consultera en priorité les sections Tests unitaires et Tests d'intégration afin de s'assurer que chaque module implémenté passe l'ensemble des jeux d'essai définis avant toute livraison.

2.2 Maîtrise d'ouvrage

La maîtrise d'ouvrage se référera principalement à la section Tests fonctionnels pour valider que le comportement de l'application correspond aux attentes exprimées dans le cahier des charges. Les tableaux de tests fonctionnels servent de base au cahier de recette.

3. Concepts de base

Les concepts suivants sont nécessaires à la compréhension des tests décrits dans ce document :

Quantification et dequantification

La quantification réduit la précision d'un signal en supprimant le bit de poids faible ($xQ[i] = (x[i] \gg 1) \ll 1$). La dequantification reconstruit les valeurs originales possibles : pour chaque échantillon quantifié $xQ[i]$, les deux candidats sont $xQ[i]$ (pair) et $xQ[i]+1$ (impair).

Histogramme de transitions (P_x)

Table de hachage comptant le nombre d'occurrences de chaque couple (valeur_précédente → valeur_suivante) dans le signal original. Chaque noeud de l'arbre dispose d'une copie locale du P_x , décrétement à chaque transition empruntée. Lorsque le compteur d'une transition atteint zéro, la branche correspondante ne peut plus être explorée.

Arbre binaire de reconstruction

Structure arborescente où chaque noeud représente une valeur candidate à un rang donné. Le fils gauche correspond à la valeur paire, le fils droit à la valeur impaire. Les feuilles atteignant le rang $n-1$ sont des solutions valides ; les noeuds sans fils valides sont marqués $dead=1$ (rouge dans SDL).

4. Tests fonctionnels

Les tests fonctionnels vérifient que l'application couvre l'ensemble du périmètre fonctionnel défini dans le cahier des charges. Ils sont définis de façon externe (boîte noire) et constituent le socle minimum du cahier de recette.

4.1 TF-01 — Lecture de fichier série

ID	Titre	Statut	Priorité
TF-01	Lecture de fichier .dat	testé	Haute
Description	Vérifier que la fonction <code>read_series()</code> lit correctement un fichier binaire de shorts et retourne les données attendues avec le bon nombre d'échantillons.		
But du test	Prouve que le module de lecture de série est fonctionnel et capable de traiter des fichiers de tailles variées sans erreur d'allocation ni de lecture.		
Contraintes	Nécessite un fichier .dat valide. Tester avec <code>x10.dat</code> , <code>x100.dat</code> , <code>x500.dat</code> etc..		
Dépendances	Aucune — test indépendant.		
Données d'entrée	Fichier binaire <code>x10.dat</code> contenant 10 shorts signés (exemple).		
Résultats attendus	Pointeur non NULL, <code>n_samples = 10</code> , valeurs cohérentes avec le signal connu.		
Critères de validation	<code>num_samples == taille_fichier / 2</code> (car un <code>short</code> occupe 2 octet) et la taille du fichier est en octet ; aucune erreur d'allocation ; valeurs dans la plage <code>[-32768, 32767]</code> . Égalité stricte valeur par valeur entre le tableau lu et les 10 valeurs attendues		

4.2 TF-02 — Quantification du signal

ID	Titre	Statut	Priorité
TF-02	Quantification 1 bit	testé	Haute
Description	Vérifier que <code>quantize_series()</code> produit une série où chaque valeur est paire (bit 0 = 0) et que la différence avec le signal original ne dépasse jamais 1.		
But du test	Prouve que la quantification est correcte : <code>xQ[i]</code> est pair et $ x[i] - xQ[i] \leq 1$ pour tout <code>i</code> .		
Contraintes	Dépend de <code>read_series()</code> . Signal d'entrée avec valeurs paires et impaires mélangées.		
Dépendances	TF-01 doit passer.		
Données d'entrée	Série <code>x</code> de 10 échantillons avec valeurs arbitraires paires et impaires (exemple).		
Résultats attendus	$xQ[i] \% 2 == 0$ pour tout <code>i</code> ; $xQ[i] == x[i]$ si <code>x[i]</code> pair ; $xQ[i] == x[i]-1$ si <code>x[i]</code> impair. chaque valeur vérifiable manuellement.		

Critères de validation	Vérification bit-à-bit : $(xQ[i] \& 1) == 0$ pour tout i ; $\text{abs}(x[i] - xQ[i]) \leq 1$. Égalité stricte valeur par valeur entre xQ obtenu et xQ attendu.
-------------------------------	--

4.3 TF-03 — Construction de l'histogramme

ID	Titre	Statut	Priorité
TF-03	Histogramme des transitions	testé	Haute
Description	Vérifier que <code>build_histogram()</code> construit correctement la table de hachage des couples (prev, curr) depuis le signal original x.		
But du test	Prouve que toutes les transitions présentes dans x sont comptabilisées et que <code>get_count()</code> retourne des valeurs cohérentes avec les occurrences réelles.		
Contraintes	Signal court (10 échantillons) permettant le comptage manuel des transitions.		
Dépendances	TF-01 doit passer.		
Données d'entrée	Signal $x = [10, 11, 10, 11, 10, 12, 10, 11, 10, 12]$ transitions connues (exemple).		
Résultats attendus	<code>get_count(10,11)=3</code> ; <code>get_count(11,10)=3</code> ; <code>get_count(10,12)=2</code> ; <code>get_count(12,10)=2</code> .		
Critères de validation	Égalité stricte entre les compteurs retournés et les occurrences comptées manuellement.		

4.4 TF-04 — Reconstruction (signal 10 échantillons)

ID	Titre	Statut	Priorité
TF-04	Dequantification signal court	testé	Haute
Description	Vérifier que <code>build_tree()</code> reconstruit exactement le signal original x à partir de xQ pour un signal de 10 échantillons. La solution unique doit correspondre à x.		
But du test	Prouve que l'algorithme de reconstruction est correct sur un cas simple où la solution est unique et vérifiable manuellement.		
Contraintes	Signal <code>x10.dat</code> avec une seule solution possible (toutes les valeurs de x sont paires, par exemple).		
Dépendances	TF-01, TF-02, TF-03 doivent passer.		
Données d'entrée	Fichier <code>x10.dat</code> , $n=10$ échantillons avec signal à solution unique.		
Résultats attendus	<code>branches_valides == 1</code> ; la solution affichée correspond exactement à x.		
Critères de validation	<code>branches_valides == 1</code> ; comparaison valeur par valeur entre la solution et x.		

4.5 TF-05 — Reconstruction (signal 100 échantillons)

ID	Titre	Statut	Priorité
----	-------	--------	----------

TF-05	Dequantification signal moyen	À tester	Haute
Description	Vérifier que <code>build_tree()</code> termine normalement sur 100 échantillons, que le programme ne boucle pas indéfiniment et que le signal original <code>x</code> fait partie des solutions trouvées.		
But du test	Prouve la correction et la terminaison de l'algorithme pour des tailles de signal réalistes.		
Contraintes	Fichier <code>x100.dat</code> . Le test peut durer quelques secondes selon le nombre de solutions.		
Dépendances	TF-01, TF-02, TF-03 doivent passer.		
Données d'entrée	Fichier <code>x100.dat</code> , <code>n=100</code> .		
Résultats attendus	Le programme se termine ; <code>x</code> figure parmi les solutions ; <code>branches_valides</code> $\geq$ 1.		
Critères de validation	Terminaison normale du programme ; présence de <code>x</code> dans la liste des solutions affichées.		

4.6 TF-06 — Noeuds morts marqués en rouge (SDL)

ID	Titre	Statut	Priorité
TF-06	Affichage noeuds morts	testé	Haute
Description	Vérifier visuellement que les noeuds sans fils valides (<code>dead=1</code>) apparaissent en rouge dans la fenêtre SDL2, tandis que les noeuds actifs sont bleus et les solutions vertes.		
But du test	Prouve que le rendu graphique distingue correctement les trois états d'un noeud : vivant (bleu), solution (vert), mort (rouge).		
Contraintes	Nécessite un environnement graphique avec SDL2. Lancer avec <code>x10.dat</code> pour voir l'arbre complet.		
Dépendances	TF-04 doit passer. SDL2 initialisé.		
Données d'entrée	Lancement du programme avec <code>x10.dat</code> , observation de la fenêtre SDL2.		
Résultats attendus	Noeuds bleus pour les noeuds internes, verts pour les feuilles solutions, rouges pour les noeuds dead.		
Critères de validation	Vérification visuelle : couleur de chaque noeud conforme à son état dead/solution/interne.		

5. Tests d'intégration

Les tests d'intégration vérifient que les différents modules du programme s'interfacent correctement. On distingue les interfaces internes (entre fichiers C) et les interfaces externes (SDL2, fichiers .dat).

NB :

On utilisera le terme interface pour désigner les fichiers.c\h du programme.

5.1 TI-01 — Interface series ↔ histogram

ID	Titre	Statut	Priorité
TI-01	Chaîne lecture → histogramme	testé	Haute
Description	Vérifier que la séquence <code>read_series()</code> → <code>build_histogram()</code> fonctionne correctement lorsque les deux modules sont enchaînés : les données lues par <code>series.c</code> sont correctement transmises à <code>histogram.c</code> via le pointeur <code>short x</code> , sans corruption ni fuite. Contrairement à TF-03 qui teste <code>build_histogram()</code> seul sur un tableau créé en mémoire, TI-01 teste l'interface réelle entre les deux modules à travers un fichier disque		
But du test	Prouve que le passage de <code>short* x</code> entre <code>series.c</code> et <code>histogram.c</code> est correct et que <code>hash_table</code> est correctement alimenté.		
Contraintes	Valgrind (outil d'analyse de fuite mémoire etc.. en C) recommandé pour détecter les fuites. Tester avec <code>x100.dat</code> .		
Dépendances	TF-01 et TF-03 doivent passer indépendamment.		
Données d'entrée	Fichier <code>x100.dat</code> lu par <code>read_series()</code> puis passé directement à <code>build_histogram()</code> — aucune manipulation du tableau entre les deux appels. Le test vérifie que le pointeur transmis est valide et que les compteurs résultants sont cohérents avec le fichier source.		
Résultats attendus	<code>hash_table</code> non NULL ; compteurs > 0 pour les transitions présentes dans <code>x</code> ; aucune fuite mémoire Valgrind.		
Critères de validation	Vérification des compteurs de quelques transitions connues ; sortie Valgrind : 0 erreurs.		

5.2 TI-02 — Interface histogram ↔ tree

ID	Titre	Statut	Priorité
TI-02	Transfert Px vers <code>build_tree</code>	À tester	Haute
Description	Vérifier que <code>copy_histogram(hash_table)</code> produit une copie indépendante correctement transmise à <code>build_tree()</code> , et que les décréments dans les Px locaux n'affectent pas <code>hash_table</code> global.		

But du test	Prouve l'isolation des Px locaux : chaque branche de l'arbre opère sur sa propre copie sans interférence avec les autres branches ni avec la table globale.
Contraintes	Inspecter hash_table avant et après build_tree() pour vérifier qu'il est intact.
Dépendances	TI-01 doit passer.
Données d'entrée	hash_table construit sur x100.dat, copie transmise à build_tree().
Résultats attendus	Les compteurs de hash_table sont identiques avant et après build_tree() ; branches_valides >= 1.
Critères de validation	Comparaison compteur par compteur de hash_table avant/après ; branches_valides > 0.

5.3 TI-03 — Interface tree ↔ display

ID	Titre	Statut	Priorité
TI-03	Rafraîchissement SDL depuis construire()	testé	Haute
Description	Vérifier que update_display() est appelé à chaque noeud visité, que global_root est correctement initialisé avant le premier appel, et que la fenêtre SDL se met à jour sans crash.		
But du test	Prouve que le passage de void* root entre tree.c et display.c est correct et que le cast vers Noeud* est sûr.		
Contraintes	Nécessite SDL2. Observer la fenêtre pendant la construction avec x10.dat.		
Dépendances	TF-06 doit passer.		
Données d'entrée	Lancement avec x10.dat, DELAY_NODE_MS=100 pour ralentir l'animation.		
Résultats attendus	La fenêtre SDL s'ouvre, l'arbre se construit visuellement noeud par noeud, aucun crash.		
Critères de validation	Absence de segfault ; affichage progressif visible ; compteur RANG X/100 s'incrémente.		

5.4 TI-04 — Libération mémoire complète

ID	Titre	Statut	Priorité
TI-04	Absence de fuites mémoire	À tester	Haute
Description	Vérifier avec Valgrind que la séquence complète read_series → build_histogram → build_tree → free_tree → free_histogram libère toute la mémoire allouée sans fuite ni double-free.		
But du test	Prouve que la gestion mémoire du programme est correcte de bout en bout, notamment les Px locaux copiés à chaque creer_fils() et libérés lors des noeuds morts ou en fin de branche.		
Contraintes	Lancer avec valgrind --leak-check=full. Désactiver SDL avec #define NO_DISPLAY.		
Dépendances	Tous les tests précédents doivent passer.		

Données d'entrée	Exécution complète avec x100.dat sous Valgrind (NO_DISPLAY activé).
Résultats attendus	Valgrind : 0 bytes perdus ; 0 erreurs d'accès mémoire ; no reachable blocks.
Critères de validation	Sortie Valgrind : 'All heap blocks were freed -- no leaks are possible'.

6. Tests unitaires

Les tests unitaires vérifient le comportement de chaque fonction de façon isolée. Ils sont organisés par module et constituent le niveau de test le plus fin.

6.1 Module histogram.c

TU-H01 — hash()

ID Test	Module	Statut
TU-H01	histogram.c — hash()	testé
Description	Vérifier que la fonction hash() retourne des indices dans [0, HASH_SIZE-1] et que deux couples identiques produisent le même indice.	
Données d'entrée	hash(100, 101) ; hash(0, 0) ; hash(-1, -1) ; hash(100, 101) appelé deux fois.	
Résultats attendus	Valeur dans [0, 10006] ; deux appels identiques retournent la même valeur.	
Critères de validation	0 <= résultat < HASH_SIZE pour tous les couples testés ; déterminisme vérifié.	

TU-H02 — insert_couple() / get_count()

ID Test	Module	Statut
TU-H02	histogram.c — insert/get	testé
Description	Vérifier que l'insertion d'un couple incrémente correctement le compteur et que get_count() retourne la valeur attendue.	
Données d'entrée	Insérer (10, 20) 3 fois, puis appeler get_count(10, 20).	
Résultats attendus	get_count(10, 20) == 3 ; get_count(10, 21) == 0.	
Critères de validation	Égalité stricte des compteurs ; retour 0 pour un couple absent.	

TU-H03 — copy_histogram()

ID Test	Module	Statut
TU-H03	histogram.c — copy_histogram()	testé
Description	Vérifier que copy_histogram() produit une copie indépendante : modifier la copie ne modifie pas l'original.	

Données d'entrée	Construire hash_table avec (10,20)=2. Copier. Décrémenter dans la copie. Lire les deux.
Résultats attendus	Original : get_count(10,20)==2 ; Copie : local_get_count(copie,10,20)==1.
Critères de validation	Indépendance totale des deux tables après modification de la copie.

TU-H04 — local_decrement()

ID Test	Module	Statut
TU-H04	histogram.c — local_decrement()	testé
Description	Vérifier que local_decrement() réduit le compteur de 1 sans passer en dessous de 0.	
Données d'entrée	local_get_count=1, appeler local_decrement une fois (attendu=0), puis encore une fois (attendu=0, pas -1).	
Résultats attendus	Après 1er décrement : count==0 ; après 2ème décrement : count==0 (pas négatif).	
Critères de validation	count >= 0 à tout moment ; pas de débordement vers les négatifs.	

6.2 Module tree.c

TU-T01 — creer_fils()

ID Test	Module	Statut
TU-T01	tree.c — creer_fils()	À tester
Description	Vérifier que creer_fils() initialise correctement le fils, copie le Px du père, décrémente la bonne transition et attache le fils au bon côté (filsG si pair, filsD si impair).	
Données d'entrée	Père avec valeur=10, Px contenant (10→20)=3. Appeler creer_fils(pere, 20, fils).	
Résultats attendus	fils->valeur==20 ; fils->pere==pere ; pere->filsG==fils ; local_get_count(fils->Px, 10, 20)==2.	
Critères de validation	Tous les champs initialisés ; décrement correct ; attachement côté gauche (20 pair).	

TU-T02 — construire() noeud mort

ID Test	Module	Statut
TU-T02	tree.c — construire() cas mort	testé
Description	Vérifier que construire() marque dead=1 un noeud pour lequel aucun candidat n'a de transition valide dans le Px local.	

Données d'entrée	Noeud avec valeur=50, Px vide (aucune transition depuis 50), rang=5, LONGEUR_MAXIMALE=10.
Résultats attendus	noeud->dead==1 ; branches_valides inchangé ; noeud reste dans l'arbre.
Critères de validation	dead==1 ; le noeud est toujours accessible depuis la racine ; branches_valides non incrémenté.

TU-T03 — construire() feuille valide

ID Test	Module	Statut
TU-T03	tree.c — construire() feuille valide	À tester
Description	Vérifier que construire() incrémente branches_valides et libère le Px quand rang == LONGEUR_MAXIMALE-1.	
Données d'entrée	Noeud au rang n-1, Px alloué, LONGEUR_MAXIMALE=10.	
Résultats attendus	branches_valides incrémenté de 1 ; noeud->Px == NULL après appel.	
Critères de validation	branches_valides == valeur_initiale + 1 ; noeud->Px == NULL.	

6.3 Module series.c

TU-S01 — read_series() fichier invalide

ID Test	Module	Statut
TU-S01	series.c — read_series() erreur	testé
Description	Vérifier que read_series() retourne NULL et affiche un message d'erreur lorsque le fichier n'existe pas.	
Données d'entrée	Appeler read_series('fichier_inexistant.dat', &n).	
Résultats attendus	Retour NULL ; message d'erreur sur stderr ; n non modifié ou mis à 0.	
Critères de validation	Retour == NULL ; pas de crash ; pas de fuite mémoire.	

TU-S02 — quantize_series()

ID Test	Module	Statut
TU-S02	series.c — quantize_series()	testé
Description	Vérifier que quantize_series() produit une série où chaque valeur est paire et correspond au plancher pair de x[i].	
Données d'entrée	x = [100, 101, 102, 103, -1, -2]. Appeler quantize_series(x, 6).	

Résultats attendus	$xQ = [100, 100, 102, 102, -2, -2]$. Toutes les valeurs paires.
Critères de validation	$(xQ[i] \& 1) == 0$ pour tout i ; $xQ[i] \leq x[i]$ pour $x[i]$ positif.

7. Vérification de la documentation

Cette section vérifie la concordance entre les procédures décrites dans la documentation et leur mise en oeuvre réelle dans le code source.

- La fonction `construire()` implémente fidèlement le pseudo-code fourni : création des fils avant libération du Px père, décrement dans la copie locale, marquage dead si aucun fils valide.
- La convention pair/impair ($\text{filsG} = \text{pair}$, $\text{filsD} = \text{impair}$) est cohérente entre `creer_fils()` et `build_tree()`.
- Les délais d'affichage (`DELAY_NODE_MS`) sont conformes aux paramètres définis dans `display.h`.
- La gestion mémoire (copie à chaque fils, libération après création) est conforme au principe décrit dans les commentaires du code.

8. Annexes

8.1 Glossaire

Px : Table de hachage locale à un noeud représentant les transitions encore disponibles sur le chemin courant.

Dequantification : Reconstruction des valeurs originales à partir d'une série quantifiée.

Noeud mort ($\text{dead}=1$) : Noeud de l'arbre pour lequel aucun fils valide n'existe — affiché en rouge dans SDL.

`branches_valides` : Compteur global du nombre de chemins ayant atteint le rang $n-1$ (solutions).

`LONGEUR_MAXIMALE` : Nombre total d'échantillons dans la série — profondeur maximale de l'arbre.

SDL2 : Simple DirectMedia Layer version 2 — bibliothèque graphique utilisée pour la visualisation graphique.

8.2 Références

- SDL2 Documentation : <https://wiki.libsdl.org/SDL2/FrontPage>
- Valgrind User Manual : <https://valgrind.org/docs/manual/manual.html>
- ISO/IEC 9899 — Norme C99

8.3 Index

- `branches_valides` — §3, §6.2 (TU-CA02, TU-CA03)
- `build_histogram` — §4 (TF-03), §5 (TI-01, TI-02), §6.1 (TU-H02)
- `build_tree` — §4 (TF-04, TF-05), §5 (TI-02, TI-03, TI-04)
- `construire` — §3, §5 (TI-03), §6.2 (TU-CA02, TU-CA03)
- `copy_histogram` — §5 (TI-02), §6.1 (TU-H03)
- `creer_fils` — §3, §6.2 (TU-CA01)
- `dead` — §3, §4 (TF-06), §6.2 (TU-CA02)
- `display / SDL2` — §4 (TF-06), §5 (TI-03)
- `hash` — §6.1 (TU-H01)
- `local_decrement` — §6.1 (TU-H04)
- `Px` — §3, §5 (TI-02), §6.1 (TU-H03), §6.2 (TU-CA01)
- `quantize_series` — §4 (TF-02), §6.3 (TU-S02)
- `read_series` — §4 (TF-01), §5 (TI-01), §6.3 (TU-S01)